

Contents

Implementation Plan: Phase R (rev-3) — Exact Bi-material Riemann Solver, Verified via seas_spatial_dyn_driver on TPV205 □ TPV31	1
Deviations from rev-2 (explicit list — REVIEW R-010)	1
Overview	2
Constraints	2
Phase 0: Inventory of artifacts produced by this plan	3
Phase R.1: Standalone bi-material flux solver	4
Phase R.2: WaveOperator dispatch — bi-material on every interior face	6
Phase R.3: seas_spatial_dyn_driver refactor — problem-agnostic via TOML	10
Phase R.4: TPV205 verification through bi-material path	14
Phase R.5: TPV31 verification — true bi-material across 1D velocity structure	16
Phase R.6: Regression + parent-plan cleanup	19
Testing Strategy	20
Risk Assessment	21
Out of Scope for This Plan	22
Cross-references	22
Open questions / decisions deferred to implementation time	23

Implementation Plan: Phase R (rev-3) — Exact Bi-material Riemann Solver, Verified via seas_spatial_dyn_driver on TPV205 □ TPV31

Status: plan (rev-3, restructured around seas_spatial_dyn_driver) **Date:** 2026-05-18 **Supersedes:** PLAN_phase_R_exact_bimaterial_riemann.md (rev-2). Rev-2 is preserved as the math/eigenvector reference and is cited by section from rev-3 below. Rev-3 differs from rev-2 in four substantive ways: 1. **No IsHomogeneous runtime fall-through.** The bi-material code path runs on EVERY interior face, including faces with identical materials on both sides. Correctness in the homogeneous limit must come from the math of the projector ($\text{matR} \cdot \text{chi} \cdot \text{matR}^{-1}$ collapsing to the upwind Godunov projector), not from a dispatch gate. Direct user direction. 2. **Verification via real SCEC benchmarks (TPV205, then TPV31), not an analytic layered-medium pulse.** Phase R.4's analytic R/T test is replaced by TPV205 (homogeneous SCEC benchmark, exercised through the bi-material path) and TPV31 (true bi-material — discontinuous 1D velocity structure crossed by the strike-slip rupture). 3. **One driver — seas_spatial_dyn_driver — handles both TPV205 and TPV31 via per-benchmark TOML configs.** The driver is refactored to be problem-agnostic; benchmark-specific knobs (boundary attributes, fault geometry, hypocenter, friction parameters, material specification) all move to TOML. This is the structural validation gate for the SAFS production driver. 4. **TPV34 and SAFS R.5 are out of scope for this plan.** Future work.

Reference implementation: SeisSol (/Users/chunhuizhao/projects/SeisSol). See rev-2 § “Mathematical formulation” for the canonical matR eigenvector packing. Rev-3 does not re-derive the math; it inherits rev-2 verbatim.

Deviations from rev-2 (explicit list — REVIEW R-010)

The following rev-2 deliverables are **intentionally dropped** in rev-3. Anything in rev-2 not listed here is inherited verbatim.

1. **WaveOperator::BimaterialFluxMode enum + setter + --bimaterial-flux {exact|average} CLI flag.** Rev-2 used these for the analytic R.4 layered-medium A/B test. Rev-3 replaces R.4 with TPV205 + TPV31 SCEC comparisons (R.4, R.5), which provide stronger correctness signal than the average-vs-exact A/B; the toggle is no longer needed and the bi-material path is the only path.
2. **BimaterialFlux::IsHomogeneous dispatch gate.** Rev-3 runs the bi-material path on every interior face (no fall-through), so the homogeneity check is unused at runtime. IsHomogeneous may still

appear as a unit-test helper but is not part of the public API.

3. **Phase R.4 analytic layered-medium R/T test.** Replaced by §R.4 (TPV205 end-to-end) and §R.5 (TPV31 end-to-end).
4. **Phase R.5 SAFS-scale acceptance + cleanup.** Deferred to a follow-up plan (see §“Out of Scope for This Plan”).

Overview

Replace the average-flux approximation at heterogeneous interior faces with the **exact linearised Riemann solution** in the SeisSol formulation (Pelties et al. 2012, §2.3 — see rev-2 § “Mathematical formulation”). Apply the bi-material formulation to **every** interior face (no homogeneous fall-through). Validate end-to-end by routing TPV205 (homogeneous, planar fault, LSW) and TPV31 (1D bi-material, planar vertical strike-slip fault, LSW) through a single refactored `seas_spatial_dyn_driver` and comparing each against its SCEC benchmark trace bank.

The “one driver, many problems” structure is itself a deliverable: it validates the driver shape SAFS production will use. Per-benchmark TOML configs (`tpv205.toml`, `tpv31.toml`) replace the hand-coded per-driver `main()` walls in `drivers/tpv205_driver.cpp`.

Constraints

Interface constraints (cannot change)

- **GodunovFlux::Interior signature and behaviour preserved verbatim.** All existing TPV / BP5 drivers (`drivers/tpv102_driver.cpp`, `drivers/tpv104_driver.cpp`, `drivers/tpv205_driver.cpp`, `drivers/seas_driver.cpp`) continue to call it byte-identically and must produce byte-identical output.
- **GodunovFluxPool API unchanged** — `Build()`, `At(e)`, `NumUniqueTriples()`, `NumElements()` keep their signatures.
- **FaultFaceFlux::EvaluateADER_LSW{,_ForcedRupture} bi-material guard preserved** (`dynamic/fault_face_flux.cpp:743`). Fault faces stay on `FaultFaceFlux`; the bi-material Riemann work touches non-fault interior faces only.
- **Phase H Stage 1 ctor / per-element CFL / GodunovFluxPool build preserved verbatim** (commit `b917fca`).
- **MaterialField API unchanged** — `Mode::Constant`, `Mode::GridFunction`, `Mode::Coefficient`; `MakeConstant`, `MakeGridFunction`, `MakeCoefficient`; `At`, `EvalAt`, `MaxCpInElement`. See `dynamic/heterogeneous_material.hpp:48–218`.

Dependency constraints

- Depends on Phase H Stage 1 (`owned_flux_pool_`, `per_elem_lmr_`, `shared_face_neighbour_material_`) — committed in `b917fca`.
- Depends on Phase H Stage 2 (per-element flux dispatch in `wave_operator.inl`’s hot loop + cross-rank `MPI_Allgatherv` exchange) — NOT yet committed. **Phase R rev-3 lands AS PART OF Phase H Stage 2** so the production tree never sees an average-flux placeholder.
- No new external libraries; uses `mfem::DenseMatrix` + its `LUSolver` for the 9×9 inverse, same as `GodunovFlux`.

Convention constraints

- File naming: `dynamic/godunov_flux_bimaterial.{hpp,cpp}` (mirrors `godunov_flux_pool` style); helper class `BimaterialFlux`.
- Test naming: `tests/unit/test_phaser_*.cpp` mirroring `tests/unit/test_phaseh_*.cpp`. Aggregate makefile target `test-phaser-*`.

- Sign / coordinate conventions (per miniapps/seas/CLAUDE.md): face normal $\hat{n}$ is the unit outward normal from Elem1; tangents $\hat{t}_1 = \text{dip}$, $\hat{t}_2 = \text{strike}$ (Tandem FaultBasis convention). Phase R inherits rotation from GodunovFlux::BuildFrame / BuildRotation / BuildRotationInverse and does not redefine them.
- All physical units SI.
- TOML configs: lowercase snake_case keys; per-benchmark configs live under <benchmark_dir>/configs/<benchmark>.toml (e.g. miniapps/seas/tpv205/configs/tpv205.toml).

Numerical / performance constraints

- **Memory bound (revised — no fall-through):** precompute (fluxLocal, fluxNeighbor) matrix pair per cell-face POV on EVERY interior face (not just heterogeneous ones). Storage = $4 \times 81 \times 8$ bytes = 2592 bytes per face $\times$ 2 sides = ~ 5 KB per face. For TPV205's 200 m mesh (~ 5 M faces) this is ~ 25 GB across all ranks — borderline. See **Risk R-5** below; mitigation is the optional `--bimaterial-flux-storage` runtime flag (NOT implemented in this plan; documented as the escape hatch).
- **Per-face dispatch cost** (runtime): two 9×9 matrix-vector products per QP on every interior face (instead of one for the homogeneous Godunov, so $\sim 2\times$ hot-loop cost on the face flux assembly). Acceptable; Phase H.1 already documented this overhead and the headline gate is correctness not performance.
- **Homogeneous-limit correctness:** when both sides have identical (λ, μ, ρ) , the bi-material Q^* formula collapses to the homogeneous upwind Godunov state (rev-2 § “Homogeneous-limit collapse”). The TPV205 acceptance gate is the END-TO-END proof — driver output through the bi-material path must match driver output through the homogeneous-only path to within tight tolerance (see R.3.T-1). Bit-exactness is NOT required because the bi-material computes Q^* via a different floating-point sequence (matrix inverse vs closed-form $A^{+/A}$ -decomposition); the error is the rounding difference between two mathematically-equal floating-point expressions.

Phase 0: Inventory of artifacts produced by this plan

The plan delivers six things, in this order:

#	Artifact	Phase
1	dynamic/godunov_flux_bimaterial.{hpp,cpp}R.1 + 9 unit tests	
2	wave_operator.inl dispatch wired so EVERY interior face routes through BimaterialFlux::ApplyPerFaceFlux	R.2
3	drivers/spatial_dyn_driver.cpp refactored to be problem-agnostic via TOML	R.3
4	tpv205/configs/tpv205.toml + working TPV205 run through seas_spatial_dyn_driver + agreement with SCEC benchmark traces and gold reference	R.4
5	tpv31/mesh/tpv31_50m.geo + tpv31/configs/tpv31.toml + working TPV31 run through seas_spatial_dyn_driver + agreement with SCEC TPV31 trace bank (acquired separately)	R.5
6	All previous TPV / BP5 binaries remain byte-exact through their existing drivers (regression gate)	R.6

Phase R.1: Standalone bi-material flux solver

Goal

A static helper class `BimaterialFlux` with a single public method that, given two materials and the face normal, builds the 9×9 `qGodLocal` and `qGodNeighbor` matrices in the face-local frame. Plus a runtime `Apply(...)` method. At the end of this phase the solver is exercised by 9 standalone unit tests; no `wave_operator.inl` wiring yet.

Math reference: rev-2 § “Mathematical formulation (SeisSol-validated)” (lines 76–242 of the rev-2 file) is incorporated by reference. The eigenvector packing (rev-2 line 80–138), the projector definition (rev-2 line 140–168), the per-side flux formula (rev-2 line 170–198), and the homogeneous-limit collapse argument (rev-2 line 200–208) all apply unchanged.

Files to Create

- `miniapps/seas/dynamic/godunov_flux_bimaterial.hpp` — header (≈ 80 LOC).
- `miniapps/seas/dynamic/godunov_flux_bimaterial.cpp` — implementation (≈ 280 LOC).
- `miniapps/seas/tests/unit/test_phaser_bimaterial_flux.cpp` — 9 unit tests (≈ 400 LOC).

Files to Modify

- `miniapps/seas/Makefile` — additive: `GODUNOV_FLUX_BIMATERIAL_SRC/OBJ, TEST_PHASER_BIMATERIAL_FLUX` .o rule, link rule for `seas_test_phaser_bimaterial_flux`, `test-phaser-bimaterial-flux` target, entry in `make test aggregate`.

Detailed Requirements

1. **API surface** (in `godunov_flux_bimaterial.hpp`):

```
namespace mfem { namespace seas {

    /// Bi-material exact linearised Riemann solver, SeisSol formulation
    /// (Pelties et al. 2012 §2.3; SeisSol
    /// src/Equations/elastic/Model/ElasticSetup.h::getTransposedGodunovState).
    class BimaterialFlux
    {
    public:
        /// Build the per-face Godunov projector matrices `qGodLocal`
        /// and `qGodNeighbor` in the FACE-LOCAL frame ( $\hat{n} \rightarrow \hat{x}$ ,  $\hat{t}_1 \rightarrow \hat{y}$ ,  $\hat{t}_2 \rightarrow \hat{z}$ ).
        ///  $Q^* = qGodLocal \cdot Q_L_{facelocal} + qGodNeighbor \cdot Q_R_{facelocal}$ .
        /// Pre-condition: neither material is acoustic ( $\mu > \epsilon$ ).
        static void BuildGodunovStateFaceLocal(real_t lam_self, real_t mu_self,
                                              real_t rho_self,
                                              real_t lam_nbr, real_t mu_nbr,
                                              real_t rho_nbr,
                                              mfem::DenseMatrix& qGodLocal,
                                              mfem::DenseMatrix& qGodNeighbor);

        /// Convenience wrapper: build qGodLocal / qGodNeighbor in face-local
        /// frame AND apply the face rotation to produce per-side flux matrices
        /// in the GLOBAL frame.
        /// fluxLocal = T · A_self_facelocal · qGodLocal · T-1
        /// fluxNeighbor = T · A_self_facelocal · qGodNeighbor · T-1
        static void BuildPerFaceFluxMatricesGlobal(
```

```

    const real_t* nor,
    const GodunovFlux& flux_self,
    const GodunovFlux& flux_nbr,
    mfem::DenseMatrix& fluxLocal,          ///< 9x9 output
    mfem::DenseMatrix& fluxNeighbor);      ///< 9x9 output

    /// Runtime per-QP apply:
    /// F_self = fluxLocal · Q_self + fluxNeighbor · Q_neighbor.
    static void ApplyPerFaceFlux(const mfem::DenseMatrix& fluxLocal,
                                const mfem::DenseMatrix& fluxNeighbor,
                                const real_t* Q_self,
                                const real_t* Q_nbr,
                                real_t* F_h_self);

};

}} // namespace

```

Note: `IsHomogeneous(...)` is removed from the public API relative to rev-2. rev-3 has no runtime homogeneity fall-through, so the helper is only useful as a precondition assertion in unit tests; if needed there, define it static inside the test file.

2. **BuildGodunovStateFaceLocal algorithm** — verbatim from rev-2 § “Phase R.1 Detailed Requirements” item 2 (rev-2 lines 416–466). Key points:
 - Compute cp_L , cs_L , cp_R , cs_R , lp_L , lp_R from the two material triples.
 - Verify non-acoustic on both sides; abort with the documented diagnostic otherwise.
 - Build $matR$ against OUR state ordering (SXX, SYY, SZZ, SXY, SYZ, SXZ, VX, VY, VZ) from `dynamic/wave_state.hpp`. Cross-check via R.1.T-1 (homogeneous-limit byte-exact) — that test fires on any row-index drift.
 - Build $\chi = \text{diag}(1, 1, 1, 0, 0, 0, 0, 0, 0)$.
 - $godunov = matR \cdot \chi \cdot matR^{-1}$ via `mfem::DenseMatrixInverse`.
 - $qGodLocal = I - godunov$, $qGodNeighbor = godunov$. Un-transposed convention (matches MFEM’s column-vector Mult semantics; rev-2 line 460–466 documents the SeisSol-vs-MFEM transposition note).
3. **BuildPerFaceFluxMatricesGlobal algorithm** — verbatim from rev-2 § “Phase R.1 Detailed Requirements” item 3 (rev-2 lines 467–483).
4. **ApplyPerFaceFlux** — verbatim from rev-2 § “Phase R.1 Detailed Requirements” item 4 (rev-2 lines 484–510).

Edge Cases to Handle

Same as rev-2 § “Phase R.1 Edge Cases” (lines 520–535) MINUS the `IsHomogeneous`-related case.

Acceptance Criteria

Reused verbatim from rev-2 except R.1.T-9 is dropped (no `IsHomogeneous` dispatch). Each criterion below maps 1:1 to a unit-test function in `tests/unit/test_phaser_bimaterial_flux.cpp`.

- **R.1.T-1 (homogeneous-limit byte-exact, the headline math gate):** for 100 random (Q_{self}, Q_{nbr}, nor) triples $\times$ 10 random (λ, μ, ρ) triples with $flux_{self} == flux_{nbr}$, the result $F = (fluxLocal + fluxNeighbor) \cdot \dots + \dots$ from `ApplyPerFaceFlux` matches `GodunovFlux::Interior(nor, Q_self, Q_nbr, F)` bit-for-bit. **Caveat:** “bit-for-bit” requires the two FP sequences to be identical; if the bi-material LU-inverse path introduces a few ULPs of drift, relax to $1e-13$ relative. Decision deferred to implementation time; document in the test file’s comment.

- ❑ **R.1.T-2 (P-wave analytic transmission):** for $\hat{n} = (1, 0, 0)$ and a right-going P-wave on L only, the interface velocity is $v_n^* = T_p \cdot v_n^L$ with $T_p = 2 Z_p^R / (Z_p^L + Z_p^R)$. Tolerance $1e-12$ for $(Z_p^L, Z_p^R) \in \{(Z, 2Z), (Z, Z/3), (Z, 5Z)\}$.
- ❑ **R.1.T-3 (P-wave analytic reflection):** same setup; reflected amplitude matches $R_p = (Z_p^R - Z_p^L) / (Z_p^L + Z_p^R)$.
- ❑ **R.1.T-4 (S-wave analytic transmission):** as R.1.T-2 but for $v_{\{t1\}}$ channel and $\mu_L \neq \mu_R$, λ arbitrary.
- ❑ **R.1.T-5 (rotation invariance):** rotate nor to $(1/\sqrt{3}) \cdot (1, 1, 1)$ and apply the same rotation to $Q_{\text{self}}, Q_{\text{nbr}}$; result differs from axis-aligned case by that exact rotation. Tolerance $1e-12$.
- ❑ **R.1.T-6 (role-swap sign symmetry):** ($\text{self}=A, \text{nbr}=B, \text{nor}=\hat{x}$) vs ($\text{self}=B, \text{nbr}=A, \text{nor}=-\hat{x}$) with matching state swap; outputs agree to round-off after outward-normal sign flip.
- ❑ **R.1.T-7 (qGodLocal + qGodNeighbor invariant):** $q\text{GodLocal} + q\text{GodNeighbor} = I$ to $1e-12$ relative.
- ❑ **R.1.T-8 (acoustic abort):** `BuildGodunovStateFaceLocal(mu_self=0, ...)` and `(mu_nbr=0, ...)` both abort via `MFEM_VERIFY`.

Dependencies

- Depends on: nothing (additive).
- Required by: R.2.

Phase R.2: WaveOperator dispatch — bi-material on every interior face

Goal

Wire the bi-material Riemann path into `WaveOperator<MeshType>` so that on a `WaveOperator` constructed via the `(MaterialField, BoundaryConfig)` ctor (Phase H Stage 1), **every** interior face — homogeneous OR heterogeneous — routes through `BimaterialFlux::ApplyPerFaceFlux`. No homogeneous fall-through.

The scalar-material `WaveOperator(MeshType&, int, real_t, real_t, real_t, const BoundaryConfig&)` ctor remains untouched and continues to use `flux_.Interior(...)` directly (the existing TPV / BP5 path). This preserves the byte-exact contract for every existing driver.

Pre-condition

Phase H Stage 2 has not yet been committed. Phase R rev-3 LANDS AS PART OF Phase H Stage 2 — they share ONE commit titled "seas/Phase H Stage 2 + Phase R rev-3: per-element bi-material Riemann dispatch (no homogeneous fall-through)". Do NOT commit Phase H Stage 2 with an average-flux placeholder and then replace it; that briefly stages inferior physics into the production tree.

Files to Modify

- `miniapps/seas/dynamic/wave_operator.hpp`:
 - Add private member (REVIEW R-004 — per-(face, side) storage; the rev-2/early-rev-3 4-entry-per-face scheme was incoherent for shared faces where `Elem2` lives on another rank):


```
// Per-interior-face precomputed flux matrices, indexed by
// (local_face_idx, side), where `side ∈ {0, 1}` selects whose
// POV: side=0 = the POV of the element returned by
// FaceElementTransformations::Elem1No on this face; side=1 =
// the POV of Elem2No. Inner array stores (fluxLocal,
// fluxNeighbor) from that POV.
//
// Population rule:
```

```

// * Fully-local interior face (both Elem1 and Elem2 on this rank):
//   BOTH side=0 and side=1 are populated, because each cell
//   assembles its own residual using its own A^self.
// * Shared face (Elem2 lives on another rank): ONLY side=0 is
//   populated. MFEM convention: on a shared-face
//   FaceElementTransformations, Elem1No is always the local
//   element; side=1 access on a shared face must abort.
//
// Populated by BuildPerFaceBimaterialFluxMatrices_; consumed at
// every Mult / ComputeFaceFluxRHS call. Indexed by local face index.
//
// Memory: 2 × 81 × sizeof(real_t) ≈ 1.3 KB per (face, side).
// Fully-local interior face uses 2.6 KB; shared face uses 1.3 KB.
std::vector<std::array<std::array<mfem::DenseMatrix, 2>, 2>>
    per_face_bimaterial_flux_;
//   per_face_bimaterial_flux_[face_idx][side][0] = fluxLocal
//   per_face_bimaterial_flux_[face_idx][side][1] = fluxNeighbor

```

– Add private helper declaration `void BuildPerFaceBimaterialFluxMatrices_();` — called from the (MaterialField, BoundaryConfig) ctor after `BuildGodunovFluxPool_` and `ExchangeBiMaterialNeighbours_`.

• `miniapps/seas/dynamic/wave_operator.inl`:

– Implement `BuildPerFaceBimaterialFluxMatrices_`:

1. Iterate over every interior face AND every shared face EXCEPT:

- * Fault faces (skip — they go through `FaultFaceFlux`; identified via existing `fault_interior_faces_ / fault_shared_faces_sets`).
- * Boundary faces (skip — handled by absorbing / free-surface BC dispatch).

2. For each remaining face, look up (λ , μ , ρ) for both sides:

- * Interior (fully-local): `per_elem_lmr_[Elem1], per_elem_lmr_[Elem2]`.
- * Shared: `per_elem_lmr_[local_elem], shared_face_neighbour_material_[shared_face_idx]`.

3. **Per-(face, side) population (REVIEW R-004):**

- * **Fully-local interior face:** call `BimaterialFlux::BuildPerFaceFluxMatricesGlobal(...)` TWICE — once with (`self=Elem1, nbr=Elem2`) storing into `per_face_bimaterial_flux_[face_idx][0]` and once with (`self=Elem2, nbr=Elem1`) storing into `per_face_bimaterial_flux_[face_idx][1]`.
- * **Shared face:** call `BuildPerFaceFluxMatricesGlobal(self=local_elem, nbr=remote_material)` ONCE; store into `per_face_bimaterial_flux_[face_idx][0][...]` (MFEM convention: local element = `Elem1` on shared `FaceElementTransformations`). Leave `[1]` default-constructed; runtime dispatch on a shared face never reads `side=1`. Optionally assert `side==0` at every shared-face dispatch site for defence in depth.

4. Log the interior-face count + total bytes consumed at rank 0.

– At EVERY `flux_.Interior(...)` call site in the dispatch — audit list via `grep -n 'flux_.Interior' wave_operator.inl` (the audit must enumerate ALL sites; rev-2 line 681 specifies “≥ 2 sites: the RK4 / `ComputeFaceFluxRHS` path AND the ADER `ComputeADER-FaceFluxRHS` path”, confirmed by the search results below):

Current `flux_.Interior` call sites (from `grep -n` at planning time):

- * `wave_operator.inl:2674, 2676` — RK4 imposed-state path (fault dispatch fallback; NOT TO BE WRAPPED — these are fault-side, not interior).
- * `wave_operator.inl:2819` — RK4 interior-face `ComputeFaceFluxRHS` (WRAP).
- * `wave_operator.inl:3316` — ADER imposed-state path (fault-side; NOT TO BE WRAPPED).
- * `wave_operator.inl:3384` — ADER interior-face flux (WRAP).
- * `wave_operator.inl:3892, 3894` — fault imposed-state ADER path (NOT TO BE WRAPPED).
- * `wave_operator.inl:4210, 4213` — **AUDIT IS A PREREQUISITE FOR R.2** (REVIEW R-012). Read ±20 lines around each line, determine whether the enclosing branch is the

interior-face DG flux path or the fault-side imposed-state path, and AMEND THIS SECTION with a final WRAP / SKIP decision before the R.2 patch is written.

- * wave_operator.inl:4354 — same; ± 20 -line audit + amend decision before R.2 lands.
- * wave_operator.inl:4825 — fault imposed-state in shared-fault fallback (NOT TO BE WRAPPED).
- * wave_operator.inl:4885 — same as 4354; ± 20 -line audit + amend decision before R.2 lands.

The audit costs minutes. Guessing wrong costs a TPV/BP5 regression caught only after R.2 hits main. Non-negotiable.

Implementation guidance: every flux_.Interior(nor, Q_self, Q_nbr, F_h) site that operates on a NON-FAULT, NON-BOUNDARY interior face becomes:

```
if (owned_flux_pool_) // heterogeneous ctor was used
{
    // `side` selects whose POV: 0 = the FaceElementTransformations::
    // Elem1No element's POV, 1 = Elem2No's. Computed at the call
    // site; for the standard MFEM face-flux loop typically
    // `const int side = (face->Elem1No == self_elem ? 0 : 1);`
    const int side = self_is_elem1 ? 0 : 1;
    const auto& flux_local_mat = per_face_bimaterial_flux_[face_idx][side][0];
    const auto& flux_nbr_mat   = per_face_bimaterial_flux_[face_idx][side][1];
    BimaterialFlux::ApplyPerFaceFlux(flux_local_mat, flux_nbr_mat,
                                     Q_self, Q_nbr, F_h);
}
else
{
    // Scalar-material ctor was used — UNCHANGED, byte-exact for TPV/BP5.
    flux_.Interior(nor, Q_self, Q_nbr, F_h);
}
```

The per-(face, side) storage lets either-POV assembly read its precomputed matrix in $O(1)$ without re-rotation.

- **CRITICAL invariant** (carries forward from rev-2 line 726–745): in MFEM's element-local DG assembly each cell computes its own face flux for its own residual. Hence the per-(face, side) store: each cell uses its own A_self and therefore sees a DIFFERENT fluxLocal. For shared faces only side=0 is populated (the local element is Elem1 by MFEM convention); the remote rank populates side=0 of its own copy of the face.

Detailed Requirements

1. **Audit all flux_.Interior(...) call sites** before writing the patch. The current list (grep -n 'flux_.Interior' wave_operator.inl at planning time) is documented above. Every interior-face site (not the fault-side imposed-state sites) gains the new dispatch.
2. **Shared-face POV symmetry verification (deferred to R.6):** wave.VerifySharedFaceBimaterialFlux() helper (analogous to VerifySharedFaultD0FDataConsistency) gathers all shared-face flux matrices to root and asserts bit-equality across the (rank A, rank B) pair that owns each shared face. Called once after R.2 lands.
3. **DR / boundary faces are SKIPPED** in BuildPerFaceBimaterialFluxMatrices_ (same as rev-2 line 773–779).
4. **Per-face memory log** at construction (REVIEW R-004 — accounts for fully-local interior using both sides, shared using one side):

```
[wave_operator] BimaterialFlux precomputation:
  interior faces processed = N_int    (2 sides each)
  shared faces processed   = N_shr    (1 side each, Elem1 = local)
```


total bytes = (2 * N_int + N_shr) * 2 * 81 * sizeof(real_t)

5. **MixedFluxMode mutual-exclusion gate (REVIEW R-002).** The current WaveOperator enforces mutual exclusion between mixed_flux_mode_ != None and use_precomputed_face_fluxes_ (dynamic/wave_operator.inl:1441-1447). Phase R's per_face_bimaterial_flux_ is similar (precomputes per-face matrices, replaces the runtime Godunov call) but with different semantics. **Extend the same mutex gate:** SetMixedFluxMode(m) with m != None AND owned_flux_pool_ != nullptr aborts with the message:

“MixedFluxMode::Adjacent / AllContinuous is incompatible with the heterogeneous WaveOperator(MaterialField, BoundaryConfig) ctor. Bi-material Riemann (Phase R) replaces the face flux at every interior face including adjacent-to-fault; combining the two would double-modify the same flux. Use the scalar ctor for MixedFlux runs OR set mixed_flux = none for bi-material runs.”

Symmetric guard in the heterogeneous ctor: if mixed_flux_mode_ != None is set before the ctor finishes wiring owned_flux_pool_, abort with the same diagnostic.

TPV205 reproduction implications: the existing TPV205 gold uses mixed_flux = adjacent. When TPV205 is rerun through the bi-material path (R.4), it MUST use mixed_flux = none. R.4.T-1 is therefore relaxed from “match the existing _mixed_flux_adjacent_ gold” to “match a NEW mixed_flux=none gold captured in R.4 step 0”.

Edge Cases to Handle

- **Fault face in a heterogeneous mesh:** skipped (existing fault_interior_faces_ / fault_shared_faces_ enumeration).
- **One side at the absorbing boundary:** covered by the existing absorbing BC dispatch; bi-material only applies to two-sided interior + shared faces.
- **Identical materials on both sides** (e.g., every TPV205 face): bi-material code path runs anyway; qGodLocal + qGodNeighbor = I collapse guarantees correctness up to LU-inverse rounding. This is the design point — see R.1.T-1 and R.4.T-1.
- **Memory budget exceeded** (R-5 below): rank-0 WARNING + recommendation to partition more aggressively.

Acceptance Criteria

- ☐ **R.2.T-1 — Existing scalar-path byte-exact regression.** All existing TPV / BP5 driver targets (make test-tpv104, make test-tpv205, make test-bp5-fault-operator, make test-bp5-integration, make test-bp5-smoke) produce BYTE-IDENTICAL output to the pre-Phase-R baseline (Commit b18bed5). Every existing driver uses the scalar WaveOperator(...) ctor — owned_flux_pool_ == nullptr — and the dispatch's else branch keeps them on flux_.Interior(...) unchanged.
- ☐ **R.2.T-2 — Heterogeneous-ctor smoke:** a 2-cell test mesh (test_phaser_dispatch_smoke.cpp) constructed via the (MaterialField, BoundaryConfig) ctor with MaterialField::MakeConstant(λ , μ , ρ) runs Mult and reaches BimaterialFlux::ApplyPerFaceFlux on the interior face (gated by a build-time SEAS_DIAG_PHASER counter).
- ☐ **R.2.T-3 — Two-layer dispatch counter:** same smoke fixture with the 2 cells assigned DIFFERENT (λ , μ , ρ) (via MaterialField::MakeCoefficient + a step function in z); confirms the per-face flux matrices differ from the homogeneous case (one matrix entry diverges by > 1e-6 relative).
- ☐ **R.2.T-4 — Memory log fires** at construction with the expected interior-face count.

Dependencies

- Depends on: Phase R.1.
- Required by: Phase R.3, R.4, R.5.

Phase R.3: seas_spatial_dyn_driver refactor — problem-agnostic via TOML

Goal

Generalise drivers/spatial_dyn_driver.cpp so the SAME binary handles SAFS, TPV205, and TPV31 by changing only the TOML config. No new per-benchmark drivers. The driver becomes the **structural validation point** for the SAFS production pipeline — if it cleanly handles three problems with different boundary attributes, fault geometries, and friction-law nuances, we have evidence the SAFS run won't need bespoke driver code.

This phase is the bulk of the rev-3 work; it converts hardcoded SAFS conventions into TOML-driven parameters.

Files to Modify

- miniapps/seas/drivers/spatial_dyn_driver.cpp — promote hardcoded values to TOML fields; add the parsing + plumbing. The deviation D-1 block (lines 16–32 of the existing driver) is removed — the heterogeneous ctor is now wired in R.2 so the Mode::Coefficient abort goes away.
- miniapps/seas/spatial/code/spatial_friction.hpp — extend SpatialFrictionConfig with the new TOML sections enumerated below.
- miniapps/seas/spatial/code/spatial_friction.cpp — parser updates.

Files to Create

- miniapps/seas/tpv205/configs/tpv205.toml — config that reproduces the existing TPV205 driver run, but routed through seas_spatial_dyn_driver.
- miniapps/seas/tpv31/configs/tpv31.toml — config for TPV31 (used in R.5).
- miniapps/seas/tests/unit/test_spatial_dyn_driver_toml.cpp — unit tests on the TOML parsing surface for the new fields.

Detailed Requirements

1. **New TOML sections** to be added to SpatialFrictionConfig:

[problem]

```
# Free-form benchmark tag — written into checkpoints' driver_tag field
# and printed at startup. No code branches keyed on this; for human
# use only.
tag = "tpv205"           # "safs" | "tpv205" | "tpv31" | ...
```

[boundary]

```
# Mesh attribute → BC class mapping. Single fault attribute; lists for
# natural (free-surface / traction-free) and absorbing.
# Values below are the ACTUAL attributes used in the corresponding
# .geo files (REVIEW R-006: the previous placeholder values were wrong).
fault_attr      = 101      # SAFS: 101. TPV205: 101 (tpv2053d_*.geo
                           # Physical Surface 101 = fault). TPV31:
                           # pick in tpv31_50m.geo; recommend 101 for
                           # cross-benchmark consistency (see R.5 step 2).
natural_attrs   = [105]    # SAFS: [102] (top free surface).
                           # TPV205: [105] (tpv2053d_*.geo Physical
                           # Surface 105 = top free + side ramp).
                           # TPV31: pick (free surface at y = 0).
absorbing_attrs = [103]    # SAFS: [103, 104] (bottom + sides).
```

```

# TPV205: [103] (single tag for all four
# outer walls; tpv2053d_*.geo Physical
# Surface 103). TPV31: pick (4-6 outer
# walls or single combined tag).

[fault_geometry]
# FaultBasis seed orientation. SAFS: (0,1,0)+(0,0,1). TPV205/TPV31:
# depend on benchmark coord system.
ref_normal = [0.0, 1.0, 0.0]
up          = [0.0, 0.0, 1.0]

# Selects which FaultGeometry ctor / friction-law dispatch to use.
# "bp5_safs" - current Phase 5a SAFS ctor with BP5Params seed.
# "tpv205_lsw" - TPV205-style: planar fault, LSW, hypocenter-centred
#               nucleation via strength reduction.
# "tpv31_lsw" - TPV31-style: planar fault, 1D bi-material, LSW with
#               cohesion taper + tau_nuke circular zone.
kind = "tpv205_lsw"

[hypocenter]
# Per-benchmark hypocenter position (m) and nucleation patch geometry.
x = 0.0
y = 7500.0
z = 0.0
nucleation_radius_m = 1400.0
nucleation_taper_m  = 600.0      # TPV31: tapers 1400-2000m to background

[material]
# Material model. When kind = "constant" the [material_constant_fallback]
# block is used (back-compat). When kind = "depth_profile_1d" the
# piecewise-1D profile under [material_profile] is parsed. When kind =
# "sidecar_hdf5" the existing sidecar bundle path is used.
kind = "constant"      # "constant" | "depth_profile_1d" | "sidecar_hdf5"

[[material_profile.layer]]      # only when kind = "depth_profile_1d"
depth_top_m    = 0.0             # y-coordinate at top of layer
depth_bot_m    = 2400.0
vp_ms          = 4050.0
vs_ms          = 2250.0
rho_kgm3       = 2580.0
interp         = "constant"      # "constant" | "linear" (linearly varies
                                # between this entry and the next).

# ... additional layers ...

```

2. BoundaryConfig plumbing — replace the hardcoded:

```

BoundaryConfig bc;
bc.fault_attr = 101;
bc.natural_attrs = {102};
bc.absorbing_attrs = {103, 104};

with:

BoundaryConfig bc;
bc.fault_attr = cfg.boundary.fault_attr;

```

```

bc.natural_attrs    = std::set<int>(cfg.boundary.natural_attrs.begin(),
                                   cfg.boundary.natural_attrs.end());
bc.absorbing_attrs = std::set<int>(cfg.boundary.absorbing_attrs.begin(),
                                   cfg.boundary.absorbing_attrs.end());

MFEM_VERIFY(bc.fault_attr > 0,
            "spatial_dyn_driver: [boundary].fault_attr must be > 0");

```

3. **FaultBasis orientation** — replace the hardcoded $\text{ref_normal} = (0, 1, 0)$, $\text{up} = (0, 0, 1)$ with $\text{cfg.fault_geometry.ref_normal}$ and $\text{cfg.fault_geometry.up}$. Validate that ref_normal is unit and up is non-parallel to ref_normal in the parser (`spatial_friction.cpp`).

4. **FaultGeometry ctor selection** by $\text{cfg.fault_geometry.kind}$:

- `bp5_safs`: existing path (the BP5 ctor with BP5Params `bp5_seed` — keep unchanged).
- `tpv205_lsw`: ALSO uses the existing BP5 ctor (TPV205 is a strike-slip planar fault; the existing BP5 geometry handling generalises; verify by running R.4.T-2 below). No new ctor needed.
- `tpv31_lsw`: same path as `tpv205_lsw`. Per-DOF material lookup is what changes (1D depth profile, see step 6 below); the geometry assembly is unchanged.

5. **Friction-law dispatch** — both TPV205 and TPV31 use LSW. The existing $\text{cfg.law} == \text{FrictionLawKind::SlipWeakening}$ path stays. LSW_FforcedRupture vs plain LSW is selected by $\text{cfg.nucleation.kind}$ (already wired at line 815–819 of the existing driver).

6. **Material handling for TPV31's 1D bi-material profile:**

- Introduce a new free helper + supporting struct in `dynamic/heterogeneous_material.hpp` (declarations) and `dynamic/heterogeneous_material.cpp` (definitions) — REVIEW R-009 + R-013:

```

/// One layer of a piecewise-1D depth-varying material profile.
/// Layers must be ordered shallow-to-deep (smaller depth_top_m first);
/// contiguous (layer[i].depth_bot_m == layer[i+1].depth_top_m,
/// possibly with a repeat at a discontinuity). `interp = "linear"`
/// linearly interpolates (vp, vs, rho) between this layer's top
/// values and the NEXT layer's top values; "constant" holds the
/// layer's values across the whole [depth_top_m, depth_bot_m] range.

```

```

struct DepthProfileLayer
{
    real_t      depth_top_m;
    real_t      depth_bot_m;
    real_t      vp_ms;
    real_t      vs_ms;
    real_t      rho_kgm3;
    std::string interp;    // "constant" | "linear"
};

```

```

/// Build a piecewise-1D depth-varying MaterialField from a TOML
/// profile. The returned wrapper owns three internal
/// `mfem::FunctionCoefficient` instances (heap-allocated, lifetime
/// tied to the returned wrapper). Use the wrapper's `MaterialField`
/// reference at all callsites; do NOT extract the raw coefficient
/// pointers separately.

```

```

///
/// Jump tie-break (REVIEW R-007): at exactly  $y = y_{\text{jump}}$ , the DEEPER
/// layer (larger- $y$  side, the "2400+" spec convention) claims the
/// point. Enforced via `if (y < layer.depth_top_m) continue;`.

```

```

struct DepthProfile1DMaterial

```

```

{
    std::unique_ptr<mfem::FunctionCoefficient> lambda;
    std::unique_ptr<mfem::FunctionCoefficient> mu;
    std::unique_ptr<mfem::FunctionCoefficient> rho;
    MaterialField                                field;
};
DepthProfile1DMaterial MakeDepthProfile1DMaterial(
    const std::vector<DepthProfileLayer>& layers,
    char depth_axis /* 'y' for TPV31 */ );

```

- The driver calls this when `cfg.material.kind == "depth_profile_1d"`. Storage of the returned wrapper must outlive wave so the `Coefficient*` references stay valid (`std::unique_ptr<DepthProfile1DMaterial>` in driver scope).

7. Hypocenter / nucleation patch generalisation:

- `ResolveForcedRupture` currently uses BP5-style depth-based partitioning. Extend to also accept a hypocenter (x, y, z) + `nucleation_radius_m` + `nucleation_taper_m` parameter set; the per-DOF $T_{\text{forced}}(r)$ becomes 0 for $r < \text{radius}$, linearly tapers to $1e9$ over $[\text{radius}, \text{radius} + \text{taper}]$.
- For TPV31, the nucleation is by **overstress** (additional shear stress in the patch), not strength-reduction. Wire `cfg.nucleation.kind = Overstress` and ensure `ResolveOverstress` returns a non-stub per-DOF $\Delta\tau$ vector matching the TPV31 formula: $\tau_{\text{nuke}}(x, y) = (4.95 \text{ MPa}) \cdot (\mu/\mu_0)$ for $r \leq 1400 \text{ m}$, taper to 0 over $[1400, 2000]$. This requires removing the existing `ResolveOverstress` ABORT-stub and replacing it with the actual implementation (referenced as a deferred follow-up in the existing driver, lines 956–964).

8. **TPV-style nucleation requires a non-stub `ResolveOverstress`** — if R.3 cannot fully wire this in one commit, an intermediate plan-internal milestone is to (a) ship R.3 with the TOML-parsed `Overstress` returning a 0-vector (so the driver runs but ignores nucleation), (b) demonstrate TPV205 (which uses `StrengthReduction`, not `Overstress`) end-to-end in R.4, (c) implement `ResolveOverstress` proper for R.5 (TPV31). This avoids blocking R.4 on a TPV31-only feature.

9. **Substep iterator selection** — `Tpv205SubStepIterator` (named for its original TPV205 use case) implements the generic LSW closed-form sub-step solve and is benchmark-agnostic. Reuse it unchanged for both TPV205 and TPV31. REVIEW R-015: a future rename to `LSWSubStepIterator` is out of scope here but recommended; in this commit add a one-line comment to the class header documenting the multi-benchmark use so the name does not mislead future readers.

10. **Output file naming:** the driver currently writes everything under `cfg.output.output_dir`. Allow `cfg.output.basename` (default = "volume") and `cfg.output.fault_basename` (default = "fault") so per-benchmark runs don't collide.

11. **Backward compatibility:** every TOML field added here has a default value matching the current SAFS hardcoded behaviour, so the existing SAFS TOML configs continue to work unchanged.

Edge Cases to Handle

- **TOML missing required fields** — abort at parse time with the field name and an example value.
- **`material.kind = depth_profile_1d` with non-monotonic layer ordering** — abort with the offending layer pair.
- **`fault_geometry.ref_normal` parallel to up** — abort.
- **Boundary attribute appears in two or more of {fault, natural, absorbing}** — abort.

Acceptance Criteria

- ❑ **R.3.T-1 — TOML round-trip:** parse `tpv205/configs/tpv205.toml` and `tpv31/configs/tpv31.toml`, then dump the parsed `SpatialFrictionConfig` and assert it matches the input field-by-field. Unit-tested in `tests/unit/test_spatial_dyn_driver_toml.cpp`.
- ❑ **R.3.T-2 — Backward-compat smoke:** existing SAFS TOML configs (whichever sample is currently in the repo under `safs/.../config/`) parse without error and produce the same `BoundaryConfig / FaultBasis ref_normal / MaterialField` as before.
- ❑ **R.3.T-3 — --dry-run on TPV205 config:** `seas_spatial_dyn_driver --config tpv205/configs/tpv205.toml --dry-run` runs to completion, prints the configured boundary attributes and material kind, exits clean.
- ❑ **R.3.T-4 — --dry-run on TPV31 config:** same, with TPV31 config. Prints the depth-profile material with each layer's (`vp`, `vs`, `rho`).

Dependencies

- Depends on: R.2 (the heterogeneous ctor must work).
- Required by: R.4, R.5.

Phase R.4: TPV205 verification through bi-material path

Goal

Run TPV205 end-to-end through `seas_spatial_dyn_driver` with `cfg.material.kind = "constant"` (so `MaterialField::Mode::Constant` — but constructed via the `(MaterialField, BoundaryConfig)` ctor so the bi-material code path is exercised). Verify against: 1. The existing `drivers/tpv205_driver.cpp` reference run (`tpv205/gold/results_mixed_flux_adjacent_p1_02_dev_job76818`). 2. The SCEC DRDG3D benchmark traces (`tpv205/benchmark_data/DRDG3D_200m_04/`).

This is the **homogeneous-limit regression gate** for the bi-material Riemann solver, but unlike rev-2 R.3.T-1, it does NOT compare a homogeneous fall-through path — it runs the full bi-material precomputation + dispatch and verifies the math collapses correctly END-TO-END.

Files to Create

- `miniapps/seas/tpv205/configs/tpv205.toml` — the canonical TPV205 config. Reproduces every parameter of `drivers/tpv205_driver.cpp` startup (`lambda`, `mu`, `rho`, BC attrs = matching the .geo mesh tagging, nucleation parameters, `mixed_flux` mode, `ader_order`, `cfl`, `tfinal`, station coordinates).
- `miniapps/seas/tpv205/scripts/compare_tpv205_traces.py` — A/B harness: reads station .dat from a `seas_spatial_dyn_driver` run + the existing `tpv205/gold/.../results/*.dat`, computes RMS error per station per field (slip-rate, slip, traction), reports pass/fail vs tolerance.
- `miniapps/seas/tests/integration/test_tpv205_spatial_driver.cpp` (or shell-out wrapper if a C++ integration test is too heavy) — minimal smoke driver: small mesh, few steps, asserts `V_max_global` stays in a known range.

Files to Modify

- `miniapps/seas/Makefile` — add `test-tpv205-spatial` target running the smoke fixture + comparison.

Detailed Requirements

0. **Capture a `mixed_flux=none` TPV205 reference** (REVIEW R-002 prerequisite). Run the existing `drivers/tpv205_driver.cpp` (unmodified) with `--mixed-flux none`, `ader-order 2`, `cfl` matching

the existing gold job_script, and write the results to `tpv205/gold/results_mixed_flux_none_p1_02_<timestamp>`. This becomes the R.4.T-1 comparison target — bi-material runs through `spatial_dyn_driver` are byte-equivalent (modulo LU rounding) to the scalar `--mixed-flux none` reference, not to the original `_mixed_flux_adjacent_gold`. The original `_mixed_flux_adjacent_gold` is preserved as the R.2.T-1 scalar-path regression target (still uses scalar ctor + `--mixed-flux adjacent`).

1. **TPV205 config** must reproduce the existing TPV205 driver byte-equivalently (REVIEW R-005). All numerical parameters **MUST** be extracted from the canonical source — `dynamic/tpv205_setup.hpp` (TPV205Params namespace) and `drivers/tpv205_driver.cpp` (CLI defaults). Do NOT assume values from spec PDFs — the driver’s actual values are authoritative (the existing test bank validates the driver, not the spec).

Specific values to extract and pin in the TOML (document the source line in TOML comments):

- Material: `TPV205Params::lambda, ::mu, ::rho` $\square$ `[material_constant_fallback]` (TPV5 / TPV205 standard is $\lambda = \mu = 32.038e9$ Pa, $\rho = 2670$ kg/m³ but verify against the header before pinning).
 - LSW friction: `TPV205Params::mu_s, ::mu_d, ::d_c` $\square$ `[friction.slip_weakening]` (TPV5 spec values are $\mu_s \approx 0.677$, $\mu_d \approx 0.525$, $d_0 \approx 0.40$ m but the driver is authoritative).
 - Boundary attributes: `TPV205Params::bc_fault_default = 101, ::bc_free_default = 105, ::bc_absorb_default = 103` $\square$ `[boundary]` (verified against `tpv205/mesh/tpv2053d_200m.geo` lines 208-211 Physical Surface tags; REVIEW R-006).
 - Mesh path: `tpv205/mesh/tpv2053d_200m.msh` (build via `gmsh -format msh22 -3 tpv205/mesh/tpv2053d_200m.geo` if not present; see `miniapps/seas/CLAUDE.md` “Known limitation — Gmsh .msh format”).
 - `cfg.numerics.mixed_flux = "none"` — **CANNOT use “adjacent” with the bi-material path** (REVIEW R-002, see §R.2 step 5). The existing TPV205 gold uses adjacent; step 0 of this phase (below) captures a NEW `mixed_flux=none` reference for the R.4.T-1 comparison.
 - `cfg.numerics.ader_order = 2`, `cfg.numerics.cfl` matching the gold run.
 - `cfg.nucleation.kind = StrengthReduction` (TPV205 uses time-weakening / forced rupture in the nucleation patch — matches the existing driver’s `LSW_ForcedRupture` dispatch).
 - Hypocenter coordinates extracted from the existing nucleation-patch code in `drivers/tpv205_driver.cpp`.
 - Output stations exactly matching the 18 stations in `tpv205/benchmark_data/DRDG3D_200m_04/`.
2. **The TPV205 run goes through R.2’s bi-material code path** (`owned_flux_pool_ != nullptr`) — not the scalar `WaveOperator(MeshT&, int,  $\lambda$ ,  $\mu$ ,  $\rho$ , bc)` path. This is the key validation: bi-material on a homogeneous problem must give the right physics.
 3. **Comparison harness** (`compare_tpv205_traces.py`):
 - For each station, load the new run’s .dat and the corresponding gold .dat from `tpv205/gold/.../results/`.
 - Compute, per field (slip-rate-1, slip-rate-2, slip-1, slip-2, traction-1, traction-2):

$$\text{rms} = \sqrt{\text{mean}((\text{new} - \text{gold})^2)}$$
and $\text{peak} = \max(|\text{new} - \text{gold}|)$.
 - PASS if $\text{rms} < 5e-3 * \text{peak}(\text{gold})$ AND $\text{peak} < 1e-2 * \text{peak}(\text{gold})$ per field per station.
 - Tolerance rationale: bi-material LU-inverse vs closed-form Godunov diverges in the $1e-13$ range per QP per step; integrated over the run a relative error in the 0.1-1% range is physically indistinguishable. If this band turns out too loose / too tight in practice, document the empirical tightening in the test file.
 - Also produces a second comparison against SCEC DRDG3D traces (`tpv205/benchmark_data/DRDG3D_200m_04/`) — this comparison’s tolerance is WIDER (multi-percent) because DRDG3D uses a different code; pass-band documented from the existing TPV205 driver’s verification baseline.

Edge Cases to Handle

- **Mesh .msh not committed** (`.gitignore` excludes `tpv205/mesh/*.msh`): the test target’s prerequisite step regenerates it via the Gmsh CLI; if Gmsh is missing the target prints a skip message with

the install command and exits clean.

- **Phase R.4 fails by > 0.5% RMS** vs gold: STOP. This is a math bug — the bi-material formulation is not collapsing to homogeneous correctly. Bisect:
 1. Re-run R.1.T-1 to confirm the single-face math is right.
 2. Re-run R.2.T-1 to confirm the scalar-ctor TPV205 driver still gives byte-exact output.
 3. If both green: bug is in BuildPerFaceBimaterialFluxMatricesGlobal (likely the $T \cdot A \cdot T^{-1}$ composition) or in the dispatch site wiring.

Acceptance Criteria

- ☐ **R.4.T-1 — TPV205 vs gold reference (HEADLINE GATE for homogeneous correctness):** `compare_tpv205_traces.py --new <new_run> --gold tpv205/gold/results_mixed_flux_adjacent_p1_02_` passes per-station per-field thresholds defined above.
- ☐ **R.4.T-2 — TPV205 vs SCEC DRDG3D reference:** `compare_tpv205_traces.py --new <new_run> --reference tpv205/benchmark_data/DRDG3D_200m_04/` passes the wider multi-percent SCEC tolerance band.
- ☐ **R.4.T-3 — Smoke test:** `make test-tpv205-spatial` runs in < 5 minutes and asserts $V_{\max_global} \in [4.0, 8.0]$ m/s (TPV205 $V_{\max}$ -at-tfinal expected window).
- ☐ **R.4.T-4 — DROPPED** (REVIEW R-011) — duplicate of R.2.T-1. The byte-exact regression on `drivers/tpv205_driver.cpp` is checked at R.2.T-1 (where the dispatch wiring lands); re-checking it at R.4 adds no signal.

Dependencies

- Depends on: R.3.
- Required by: R.5.

Phase R.5: TPV31 verification — true bi-material across 1D velocity structure

Goal

Run TPV31 end-to-end through `seas_spatial_dyn_driver` with `cfg.material.kind = "depth_profile_1d"` (the discontinuous 1D velocity structure from the TPV31 spec). Verify against the SCEC TPV31 trace bank.

This is the **bi-material correctness gate** — the only place in the rev-3 plan where the bi-material code path is exercised on a problem with actual material contrasts that match analytic expectations from the underlying physics. R.4's TPV205 is homogeneous; R.5's TPV31 has 3 distinct material layers separated by 2 horizontal discontinuities at $y=2400\text{m}$ and $y=5000\text{m}$ (with a third “soft” boundary at $y=10000\text{m}$ and continuous transitions elsewhere).

Files to Create

- `miniapps/seas/tpv31/mesh/tpv31_50m.geo` — Gmsh geometry for the TPV31 model volume: half-space, fault at $z=0$, mesh refinement zone around the fault. 50 m resolution as requested by the SCEC spec. Tagged with the boundary attributes documented in `tpv31/configs/tpv31.toml`.
- `miniapps/seas/tpv31/configs/tpv31.toml` — canonical TPV31 config. Encodes the 1D velocity profile, the stress tensor at each depth (σ_{11} , σ_{22} , σ_{33} , σ_{13} from spec page 6, all scaled by μ/μ_0), the 1400m / 2000m nucleation patch, the LSW parameters ($\mu_s = 0.580$, $\mu_d = 0.450$, $d_0 = 0.18$ m, depth-dependent cohesion).
- `miniapps/seas/tpv31/scripts/compare_tpv31_traces.py` — A/B harness against the SCEC TPV31 reference trace bank (acquisition step below).
- `miniapps/seas/tpv31/benchmark_data/` — SCEC reference traces, downloaded from strike.scec.org (the TPV31 result-submission area). Step 1 of this phase.

Files to Modify

- `miniapps/seas/Makefile` — add `test-tpv31-spatial` target.
- `miniapps/seas/spatial/code/spatial_friction.hpp` (REVIEW R-008):
 - Extend `StressSourceKind` enum with new value `DepthProportionalToShearModulus`.
 - Add `struct DepthProportionalStressSpec { real_t sigma_xx_per_mu, sigma_yy_per_mu, sigma_zz_per_mu, sigma_xy_per_mu, sigma_yz_per_mu, sigma_xz_per_mu; real_t mu_ref_pa; };` and a member `DepthProportionalStressSpec depth_proportional;` on `StressSpec`.
- `miniapps/seas/spatial/code/spatial_friction.cpp` (REVIEW R-008):
 - Parse `[stress.depth_proportional]` TOML block (6 scaled-component fields in MPa, internally multiplied by $1e6$; plus `mu_ref_pa` defaulting to $32.03812032e9$ per TPV5/TPV31 spec).
 - Map the TOML string `cfg.stress.kind = "depth_proportional"` to the new `StressSourceKind::DepthProportionalToShearModulus` enum value.
- `miniapps/seas/spatial/code/spatial_stress.{hpp,cpp}` (REVIEW R-008):
 - Add class `DepthProportionalToShearModulusStressSource` mirroring `ConstantTensorStressSource`'s public interface. Its `ApplyToFaultGeometry(...)` (or equivalent) evaluates $\mu(y)$ at each fault DOF via the active `MaterialField` and scales each component by $\mu(y) / \mu_ref_pa$. Required because TPV31 stress is depth-dependent (per spec) and proportional to local shear modulus.

Detailed Requirements

1. **Step 1 (acquisition, before any code):** download the TPV31 SCEC reference traces from strike.scec.org/cvws/cgi-bin/cvws.cgi?problem=31 (or via the cvws data API) into `miniapps/seas/tpv31/benchmark_data/`. Per SCEC spec p. 12, TPV31 has **30 on-fault stations + off-fault stations + the rupture-time contour file**. Submit the full set of stations (not a subset). Document the exact source URL in `tpv31/benchmark_data/README.md`. If acquisition fails (offline / login required), STOP and ask user for an alternate source — DO NOT proceed without reference data.
2. **Mesh generation** (`tpv31/mesh/tpv31_50m.geo`):
 - Model volume: a box surrounding the half-space. Half-space is $y \geq 0$ (per spec, y increases with depth). Outer box dimensions: **`[-50, 50] × [0, 50] × [-50, 50]` km** (REVIEW R-001).
 - **Reflection-time budget** (REVIEW R-001): deepest-layer $V_p = 6500$ m/s (spec p. 4); SCEC $t_{final} = 15.0$ s (spec p. 9, NOT 12 s). Round-trip reflection from an absorbing wall at distance d arrives at $t = 2d / V_p$; for a reflection-free 15 s window we need $d \geq 49$ km. Choose 50 km, **OR** set `cfg.numerics.use_pml = true` and shrink the box to 25–30 km with a 5 km PML zone. Document the choice in the `.geo` header comment.
 - Fault: planar at $z = 0$, $x \in [-15, 15]$ km, $y \in [0, 15]$ km.
 - Free surface at $y = 0$ (per spec: “fault reaches the earth’s surface”).
 - Mesh refinement: 50 m near the fault, ramping out to 500 m at the absorbing boundary.
 - **Boundary attribute conventions** (REVIEW R-014, must match `tpv31.toml`). Use this scheme for cross-benchmark consistency with TPV205 (101/103/105) and SAFS (101/102/103/104):
 - `Physical Surface(101)` = fault plane ($z = 0$, $x \in [-15, 15]$ km, $y \in [0, 15]$ km)
 - `Physical Surface(102)` = free surface ($y = 0$ plane)
 - `Physical Surface(103)` = absorbing — four vertical outer walls ($x = \pm d$, $z = \pm d$)
 - `Physical Surface(104)` = absorbing — bottom ($y = +box_extent$)Document at the top of `tpv31_50m.geo` with a coordinate-frame diagram matching spec p. 3.
3. **TOML config** (`tpv31/configs/tpv31.toml`):
 - **Time** (REVIEW R-001): `cfg.time.tfinal = 15.0` (per SCEC spec p. 9, NOT 12.0). PML enable flag: `cfg.numerics.use_pml = false` if the 50 km box from step 2 is used; `true` if the 30 km box + PML zone is used.

- **Fault basis** (REVIEW R-003): `[fault_geometry] ref_normal = [0.0, 0.0, 1.0], up = [0.0, 1.0, 0.0]` (NOT the SAFS $(0, 1, 0) + (0, 0, 1)$ default — see step 4 for the sign-convention derivation).
- **Material-profile layers**: all 8 entries from the spec page 4 table. For the discontinuous interfaces at $y=2400/5000/10000$, encode TWO adjacent `[[material_profile.layer]]` entries — one for $y \in [\text{prev_bot}, \text{jump})$ (shallow side) and one for $y \in [\text{jump}, \text{next_top})$ (deep side).
- **Tie-break at jump** (REVIEW R-007): for evaluation at exactly $y = y_{\text{jump}}$, use the **DEEPER layer** (larger y , the “2400+” side in spec notation). Enforced in `MakeDepthProfile1DMaterial` via `if (y < layer.depth_top_m) continue;` so the layer with `depth_top_m == y_jump` claims the point. Document the rule in the helper’s docstring; it must match the rule used for cohesion $C_0(y)$ so a single y_{jump} DOF receives consistent material+friction.
- **Initial stress block** (REVIEW R-008): use the new `[stress] kind = "depth_proportional"` value (added in the spatial_stress extension below) with `[stress.depth_proportional]` carrying the 6 scaled components in MPa + $\mu_{\text{ref_pa}} = 32.03812032\text{e9}$ (TPV5/TPV31 reference shear modulus). Component values per spec p. 6: $\sigma_{xx_per_mu} = -60$, $\sigma_{yy_per_mu} = 0$, $\sigma_{zz_per_mu} = -60$, $\sigma_{xy_per_mu} = 0$, $\sigma_{yz_per_mu} = 0$, $\sigma_{xz_per_mu} = +30$ (sign convention pinned by R-003 in step 4).
- **Nucleation**: `hypocenter = (0, 7500, 0)`, `radius = 1400`, `taper = 600`, `kind = Overstress`.
- **Friction**: LSW with $\mu_s = 0.580$, $\mu_d = 0.450$, $d_0 = 0.18$, depth-dependent cohesion $C_0(y) = (0.000425 \text{ MPa/m}) \cdot \max(0, 2400 - y)$.

4. ResolveOverstress implementation (replaces the stub, REVIEW R-003):

- Per-DOF: compute $r = \sqrt{x^2 + (y - 7500)^2}$. (TPV31 coords: x = along strike, y = depth, z = fault-normal; fault at $z = 0$.)
- Magnitude: $\Delta\tau_{\text{magnitude}} = 4.95 \text{ MPa} \cdot (\mu(y) / \mu_0)$ for $r \leq 1400$; cosine taper to 0 over $[1400, 2000]$; 0 otherwise.
- **Direction — sign convention pinned to TPV31 spec + FaultBasis** (REVIEW R-003):
 - TPV31 spec p. 6: $\sigma_{13} = +30 \text{ MPa}$ = “right-lateral shear on the planar fault”. Right-lateral with the fault at $z=0$ and `ref_normal` toward $+\hat{z}$ means `Elem1` (the $+\hat{z}$ side) slips in the $-\hat{x}$ direction.
 - With the TPV31 TOML `ref_normal = [0, 0, 1]`, `up = [0, 1, 0]` (step 3 above), `FaultBasis::Compute` produces `tangent2 = strike = (+1, 0, 0)` and `tangent1 = dip = (0, +1, 0)` (downward, since y increases with depth).
 - The pre-stress $\sigma_{13} = +30 \text{ MPa}$ projects onto our `tangent2` ($+\hat{x}$) as $\tau_{2_0} = -30 \text{ MPa} \cdot (\mu/\mu_0)$ (SIGN FLIP: our `tangent2` is $+\hat{x}$ but right-lateral on the $+\hat{z}$ side is $-\hat{x}$).
 - Therefore $\Delta\tau_{\text{strike}} = -\Delta\tau_{\text{magnitude}}$, $\Delta\tau_{\text{dip}} = 0$, $\Delta\sigma_n = 0$. Pre-stress and nucleation increment carry the SAME SIGN — both push the fault right-lateral.
 - **Validation gate**: the new R.5.T-5a sign-convention test (added below) MUST pass before R.5.T-4 trace comparison runs. If it fails, flip the sign of BOTH τ_{2_0} projection AND $\Delta\tau_{\text{strike}}$ together (preserves the spec’s right-lateral semantics; only the projection sign changes) and rerun. Document the final sign in the TOML.

5. Comparison harness (compare_tpv31_traces.py):

- Same per-field RMS/peak structure as R.4.
- Pass-band: tolerance similar to what existing SCEC SEAS / DR benchmarks accept (~5–10% peak relative error vs reference). Documented from the literature at implementation time.

Edge Cases to Handle

- **SCEC TPV31 reference data is not openly downloadable** (login wall): STOP and ask user for the data path before proceeding.

- **Mesh discontinuities don't align with material layers:** per the SCEC spec the velocity is piecewise-constant + piecewise-linear with explicit interface depths; the mesh need not align with those depths — `mfem::FunctionCoefficient` correctly samples the piecewise function at each QP without needing mesh-aligned interfaces. Document this in the mesh .geo header comment.
- **Per-DOF stress consistency at material jumps:** a fault QP on the $z = 0$ plane that happens to lie exactly at $y = 2400$ has an ambiguity in the local μ . Resolve by always evaluating μ at $y + \epsilon$ (i.e., the deeper side); document.

Acceptance Criteria

- ☐ **R.5.T-1 — Acquisition (gating):** `tpv31/benchmark_data/` contains the SCEC TPV31 reference traces (≥ 4 on-fault stations). README documents source.
- ☐ **R.5.T-2 — Mesh and config land:** `make test-tpv31-spatial` runs to completion (5–15 minutes on 8 ranks) and produces `station.dat` files at all stations matching the SCEC reference station list.
- ☐ **R.5.T-3 — Rupture nucleates and propagates:** rank-0 log reports `V_max_global > 1 m/s` at some $t < 4$ s (rupture has initiated and accelerated past the nucleation patch).
- ☐ **R.5.T-4 — Trace comparison vs SCEC TPV31 reference:** `compare_tpv31_traces.py` passes the documented per-field tolerance band on all on-fault stations. If it fails by $> 2\times$ the band on ANY station, STOP and bisect: this is either a bi-material math bug, a stress-projection bug, or a friction-parameter bug. Diagnose via R.5.T-5 (homogeneous TPV31 control) before changing the bi-material code.
- ☐ **R.5.T-5 — Homogeneous TPV31 control (sanity check):** same TPV31 config but with `material.kind = constant` and the depth-averaged (λ, μ, ρ) — re-run, confirm the rupture still propagates and produces SIMILAR (not bit-identical) traces to R.5.T-4. Documents the homogeneous baseline so R.5.T-4 failures can be attributed to material contrast vs other bugs.
- ☐ **R.5.T-5a — Sign convention pre-check (gating, REVIEW R-003):** a 10-step homogeneous TPV31 control run (R.5.T-5 setup) with the σ_{13} pre-stress + nucleation patch applied produces slip on the $+z$ side of the fault in the $-\hat{x}$ direction (right-lateral, per spec p. 6). Verify by reading `dof_data[hypocenter].V2` at step 10 and asserting $V2 < 0$ (in our `tangent2 = + $\hat{x}$` convention, right-lateral slip on the $+z$ side = $-\text{tangent2}$ motion). If this gate fails, flip the sign on BOTH `tau2_0` projection AND `$\Delta\tau_{\text{strike}}$` together (preserves the spec's right-lateral semantics; only the projection sign flips) and rerun. Document the final sign in `tpv31.toml`. **R.5.T-4 trace comparison MUST NOT RUN until R.5.T-5a passes** — otherwise a sign-flipped rupture will silently fail R.5.T-4 by $|2\times \text{ref}|$ error and waste a debugging cycle.
- ☐ **R.5.T-6 — Bulk velocity off-fault reflection check:** at an off-fault station above the $y=2400$ interface, the SH-wave time-trace shows an additional arrival (the reflection from the depth discontinuity) at the predicted $t = 2 \cdot \text{depth} / V_s$; absent in the R.5.T-5 homogeneous control. Empirical proof the bi-material physics is firing.

Dependencies

- Depends on: R.4 (the driver structure must already be working on a homogeneous problem).
- Required by: nothing in this plan (R.5 is the closing phase for rev-3).

Phase R.6: Regression + parent-plan cleanup

Goal

Confirm no existing driver regressed, and update the parent plans to reflect that the bi-material Riemann work is no longer “future work”.

Files to Modify

- `miniapps/seas/safs/project_7.0_alternative/document/spatial_dynamic_rupture_plan.md`
— replace the “Out of Scope item 1 (exact bi-material Riemann solver)” caveat with a cross-reference

to this rev-3 plan.

- `miniapps/seas/safs/project_7.0_alternative/document/heterogeneous_material_plan.md` (if present) — update Phase 3 Detailed Req. 6 to point at this plan.
- `miniapps/seas/CLAUDE.md` — append a note: “Bi-material Riemann: exact linearised solver per Pelties et al. 2012; runs on every interior face of any `WaveOperator` constructed via the `(MaterialField, BoundaryConfig)` ctor. Verified on TPV205 (homogeneous regression) and TPV31 (1D bi-material). See `PLAN_phase_R_exact_bimaterial_riemann_rev3.md`.”

Detailed Requirements

1. Run the full unit-test suite (`make test`); confirm zero regressions.
2. Run the existing TPV/BP5 driver targets (`make test-tpv102-local`, `make test-tpv104`, `make test-tpv205`, `make test-bp5-fault-operator`, `make test-bp5-integration`, `make test-bp5-smoke`); confirm bit-identical output to commit `b18bed5`.
3. Run `wave.VerifySharedFaceBimaterialFlux()` on a 4-rank TPV31 fixture; assert zero bit-mismatches.

Acceptance Criteria

- ☐ **R.6.T-1 — Full `make test` green.**
- ☐ **R.6.T-2 — TPV / BP5 byte-exact** vs `b18bed5`.
- ☐ **R.6.T-3 — Shared-face POV symmetry** zero bit-mismatch on 4-rank TPV31.
- ☐ **R.6.T-4 — Parent plans updated.**

Dependencies

- Depends on: R.5.
- Required by: nothing.

Testing Strategy

Commit	Phase(s)	What it catches	Cost
#1	R.1	Math errors in the standalone solver (<code>matR</code> packing, projector inversion, eigenvector row indices). 8 unit tests including R.1.T-1 (homogeneous-limit byte-exact against <code>GodunovFlux::Interior</code>).	minutes
#2	R.2 (with Phase H Stage 2)	Dispatch wiring + drift in TPV/BP5 (scalar-ctor) output. R.2.T-1 is the headline scalar-path regression.	minutes
#3	R.3	Driver refactor — <code>seas_spatial_dyn_driver</code> becomes problem-agnostic. TOML parsing + dry-run smoke on TPV205 and TPV31 configs.	minutes
#4	R.4	TPV205 end-to-end through bi-material path matches the gold reference + SCEC DRDG3D traces. HEADLINE homogeneous-correctness gate at integrated scale.	~10–30 min per ranks=8 run

Commit	Phase(s)	What it catches	Cost
#5	R.5	TPV31 end-to-end matches SCEC TPV31 trace bank. HEADLINE bi-material correctness gate.	5–15 min per ranks=8 run
#6	R.6	Cross-check that nothing else regressed; parent plans updated.	minutes

Strict ordering: each commit requires the previous to be green. Phase R.2 + Phase H Stage 2 share the same commit so the production tree never sees an average-flux placeholder for heterogeneous problems.

Risk Assessment

Things that have caused real bugs in similar code

1. **State-vector index ordering mismatch with SeisSol's matR** (rev-2 risk 1, reproduced). Our state order (SXX, SYY, SZZ, SXY, SYZ, SXZ, VX, VY, VZ) differs from SeisSol's (σ_{xx} , σ_{yy} , σ_{zz} , σ_{xy} , σ_{yz} , σ_{xz} , u , v , w) in the off-diagonal stress row indices; build matR against OUR state ordering in `dynamic/wave_state.hpp`. Mitigation: R.1.T-1 catches any swap on random (λ , μ , ρ , nor , Q).
2. **Transposed-vs-untransposed Godunov projector** (rev-2 risk 2). SeisSol stores the transposed form for left-mult kernels; MFEM uses column-vector right-mult. R.1.T-1 row/col swap = this is the culprit.
3. **POV asymmetry in shared-face matrices** (rev-2 risk 3). Each rank that owns a shared face must build the same matrices. `DenseMatrixInverse`'s LU pivoting CAN re-order operations; verify via R.6.T-3 which bit-compares across the rank pair.
4. **Homogeneous-limit drift through LU rounding** (NEW relative to rev-2). Because we no longer have an `IsHomogeneous` fall-through, TPV205 runs through the LU-inverse path. If R.4.T-1 fails by $>$ the documented 0.5% RMS band, the bi-material formulation is NOT collapsing to homogeneous correctly OR LU rounding is producing systematic error. Diagnose:
 - Add a `BimaterialFlux::SelfDiagnostic_HomogeneousIdentity(...)` static method that, given a triple, builds the bi-material matrices and asserts (`fluxLocal + fluxNeighbor`) == `flux_.Ax()` (rotated to global) to $1e-12$; call it from R.4's test setup before the actual run. If THAT fails, the math has a bug at scale (likely in the rotation composition); if THAT passes but R.4.T-1 still fails, the integrated dispatch is wrong.
5. **Memory blowup from precomputing on every interior face** (NEW relative to rev-2). Estimated 5 KB per face $\times$ $O(M)$ faces. For TPV205 200m mesh on 8 ranks ($\sim 625k$ faces per rank) = ~ 3 GB per rank. Likely OK; if rank-0 log reports > 8 GB, document as a follow-up to introduce the runtime-compute alternative (rev-2 R-5 mitigation).
6. **TPV/BP5 byte-exact regression from R.2 wiring** (rev-2 risk 6). Headline R.2.T-1 catches this.
7. **TPV31 SCEC reference data unavailable** (NEW). R.5.T-1 is a gating prerequisite. If acquisition fails, STOP and ask user.
8. **TOML schema explosion** (NEW). Adding `[boundary]`, `[fault_geometry]`, `[hypocenter]`, `[material_profile]` blocks to `SpatialFrictionConfig` adds parser surface area. Mitigation: R.3.T-1 + R.3.T-2 (round-trip + back-compat) on every change.
9. **Driver behavioural drift between SAFS and TPV** (NEW). The driver now has internal conditional behaviour keyed on TOML config rather than per-driver source code. Mitigation: the

Tpv205SubStepIterator + AdvanceADERWithSubStep_Spatial machinery is REUSED unchanged across both — the per-benchmark differences live in TOML (BC attrs, material, friction params, nucleation kind) and in the MaterialField construction, not in the time loop.

Known tricky areas in existing code

- The fault-face dispatch in `wave_operator.inl` (R-016 / R-1600 / R-1601) is the most intricate piece of the file. Phase R touches only the NON-fault interior-face branch; the audit list in R.2's Files-to-Modify enumerates the exact sites.
- The existing `drivers/spatial_dyn_driver.cpp` has 10+ deviation blocks (D-1, D-2, D-3, etc.) documented at lines 13–46. R.3 should close D-1 (the heterogeneous ctor gap) and document the others' status; D-2 and D-3 remain as-is.
- `tpv205/mesh/*.msh` is `.gitignore'd`; the test target must regenerate via Gmsh. Tests on machines without Gmsh installed must skip cleanly.

Out of Scope for This Plan

1. **TPV32, TPV33, TPV34, other TPV bi-material benchmarks.** TPV34 specifically is deferred per user direction (2026-05-18); TPV31 alone is the bi-material validation for this plan.
2. **SAFS-scale acceptance test on CVM-H sidecar material** (rev-2 R.5). Deferred — a follow-up plan will gate on TPV31 success.
3. **Bi-material at fault faces** (`FaultFaceFlux::Evaluate*`). Fault interfaces carry friction law + slip DOF and are physically distinct.
4. **Anisotropic material** (full elasticity tensor). SAFS is isotropic.
5. **Acoustic regions** ($\mu = 0$). SAFS-rejected upstream; bi-material aborts on acoustic input per R.1 documented diagnostic.
6. **GPU port.**
7. **Per-face precomputation FALLBACK to per-QP runtime when memory budget is exceeded** (R-5 mitigation in rev-2). Documented but not implemented; a follow-up flag `--bimaterial-flux-storage runtime` would add the per-QP path.
8. **Adaptive mesh refinement / non-conforming faces.** Same as parent plans.

Cross-references

- `PLAN_phase_R_exact_bimaterial_riemann.md` (rev-2) — math reference (eigenvector packing, projector definition, per-side flux, homogeneous limit, references).
- `spatial_dynamic_rupture_plan.md` Phase H (heterogeneous WaveOperator).
- `heterogeneous_material_plan.md` Phase 3 (MaterialField — the legacy average-flux note that this plan replaces).
- `miniapps/seas/CLAUDE.md` “Known limitation — Gmsh `.msh` format” (mesh format gate for SCEC `.msh` files).
- `dynamic/godunov_flux.{hpp, cpp}` (the homogeneous Godunov flux this work extends).
- `dynamic/godunov_flux_pool.{hpp, cpp}` (the per-element flux cache from Phase H.1).
- `dynamic/wave_operator.{hpp, inl}` (the dispatch site this work wires).
- `drivers/spatial_dyn_driver.cpp` (the binary that runs TPV205 + TPV31 + SAFS).
- `drivers/tpv205_driver.cpp` (the reference TPV205 driver this work eventually replaces for TPV205).
- `tpv205/benchmark_data/DRDG3D_{100m, 200m}_04/` (SCEC TPV205 reference traces).
- `tpv31/benchmark_document/TPV31_32_Description_v03.pdf` (SCEC TPV31 spec).

SeisSol reference paths (read at planning time, rev-2)

- /Users/chunhuizhao/projects/SeisSol/src/Equations/elastic/Model/ElasticSetup.h:80–166 — getTransposedGodunovState (canonical bi-material Riemann builder).
- /Users/chunhuizhao/projects/SeisSol/src/Initializer/CellLocalMatrices.cpp:200–355 — initializeCellMatrices (per-cell-face init pattern mirrored by BuildPerFaceBimaterialFluxMatrices_).
- /Users/chunhuizhao/projects/SeisSol/codegen/kernels/aderdg.py:227–246 — computeFluxSolverLocal / computeFluxSolverNeighbor (auto-generated kernel; BuildPerFaceFluxMatricesGlobal produces the equivalent composition).
- /Users/chunhuizhao/projects/SeisSol/src/Model/Common.h:33 — testIfAcoustic(mu) (acoustic-input rejection threshold).

Open questions / decisions deferred to implementation time

1. **R.1.T-1 tolerance:** bit-for-bit byte-exact, or relaxed to 1e-13 relative? Decide at first failing run. The bi-material LU path's FP sequence is mathematically equal to but not bit-identical to GodunovFlux::Interior's closed-form path; some ULPs of drift are expected.
2. **R.4 trace comparison tolerance:** the documented 0.5% RMS band is an initial estimate. Tighten or loosen based on empirical run; document the final value in the test.
3. **R.3 step 8 (overstress ABORT-stub):** ship R.3 with the stub still active and a clear "TPV31 requires R.5 to land first" message? Or fill in the resolver in R.3? Recommendation: fill in R.3 so R.5 only needs to plug in the TPV31-specific spatial profile; this avoids two passes over ResolveOverstress.
4. **Mesh attribute conventions for TPV205:** audit the tpv2053d_200m.geo to extract the actual fault / free / absorbing attribute IDs. R.4 step 1 documents this audit; the values may differ from the rough estimates in the TOML template here.
5. **Whether to also include TPV32** (continuous 1D velocity structure — variant of TPV31). TPV31's discontinuous structure is the stronger bi-material test; TPV32 would mostly re-validate the linear-interpolation in MakeDepthProfile1DMaterial. Recommended to defer.